Full Stack Web Development DHATRI

Lab Sheet 4 — Node.js ROLL NO- 18

TASK 1: Custom Modules

sum.js

```
// sum.js
// Custom module exporting a single function that adds two numbers.
function sum(a, b) {   return a + b;
}
module.exports = sum;
```

even-odd.js

```
// even-odd.js
// Custom module exporting a function that reports whether a number is even or odd.
function evenOdd(n) {
  return n % 2 === 0 ? "even" : "odd";
}
module.exports = evenOdd;
```

app1.js

```
// app1.js
// Imports both custom modules and calls them with sample values.
const sum = require("./sum");
const evenOdd = require("./even-odd");

console.log("---- sum.js ----");
console.log("sum(4, 7) =", sum(4, 7));
console.log("sum(-3, 10) =", sum(-3, 10));

console.log("\n---- even-odd.js ----");
console.log("evenOdd(4) =", evenOdd(4));
console.log("evenOdd(7) =", evenOdd(7));
console.log("evenOdd(0) =", evenOdd(0));
```

Terminal Output

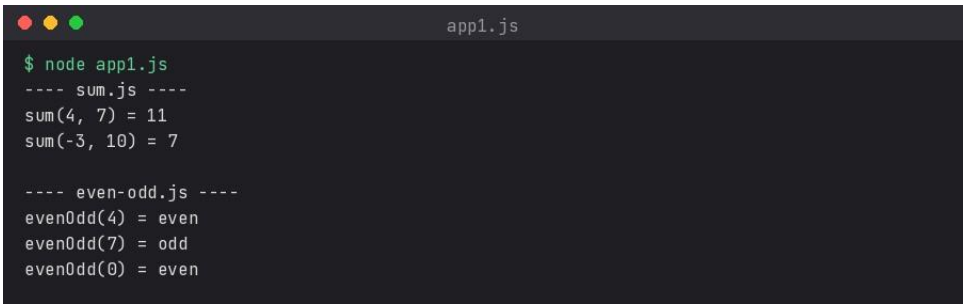


Fig 1: app1.js importing both custom modules and calling them with sample values.

TASK 2: File System — CRUD on a Text File

file-ops.js

```
// file-ops.js
// Basic CRUD on a text file using the fs module (sync methods for simplicity).
const fs = require("fs"); const path = require("path");

const studentFile = path.join(__dirname, "student.txt");
const profileFile = path.join(__dirname, "profile.txt");

// 1. CREATE + WRITE — create student.txt and write name + roll number
fs.writeFileSync(studentFile, "Name: Kanishka Sharma\nRoll No: 26\n");
console.log("student.txt created and name/roll number written.");

// 2. APPEND — add course name
fs.appendFileSync(studentFile, "Course: Full Stack Web Development\n");
console.log("Course name appended to student.txt.");

// 3. READ — read and print full content
const content = fs.readFileSync(studentFile, "utf8"); console.log("\n---
- student.txt content ----");
console.log(content);

// 4. RENAME — student.txt -> profile.txt
fs.renameSync(studentFile, profileFile);
console.log(`File renamed: student.txt -> profile.txt`);
console.log("profile.txt exists:", fs.existsSync(profileFile));
```

Terminal Output



Fig 2: student.txt created, written, appended, read back, then renamed to profile.txt.

TASK 3: Directory Operations

dir-ops.js

```
// dir-ops.js
// Creates a folder, creates files inside it, lists them, then deletes one.
const fs = require("fs"); const path = require("path");

const uploadsDir = path.join(__dirname, "uploads");

// 1. Create folder "uploads" if
(!fs.existsSync(uploadsDir)) {
fs.mkdirSync(uploadsDir);
  console.log("Created folder:", uploadsDir);
}

// 2. Create 3 empty files inside it
const files = ["file1.txt", "file2.txt", "file3.txt"];
files.forEach((file) => {
  fs.writeFileSync(path.join(uploadsDir, file), "");
  console.log("Created empty file:", file);
});

// 3. List all files in the folder
console.log("\n---- Files inside uploads/ ----");
console.log(fs.readdirSync(uploadsDir));

// 4. Delete one of the files
fs.unlinkSync(path.join(uploadsDir, "file2.txt"));
console.log("\nDeleted file2.txt");

console.log("\n---- Files inside uploads/ after delete ----");
console.log(fs.readdirSync(uploadsDir));
```

Terminal Output



Fig 3: uploads/ folder created with 3 files, listed, then file2.txt deleted and the folder listed again.

TASK 4: Path Module

path-demo.js

```
// path-demo.js
// Demonstrates common path module operations on a hardcoded file path.
const path = require("path");

const filePath = "/home/user/data/report.pdf";

console.log("Full path:", filePath);
console.log("Directory name (dirname):", path.dirname(filePath));
console.log("Base name (basename):", path.basename(filePath));
console.log("Extension (extname):", path.extname(filePath));

// Absolute path resolved from a relative one
const relativePath = "./reports/2026/summary.pdf";
console.log("\nRelative path:", relativePath);
console.log("Resolved absolute path:", path.resolve(relativePath));
```

Terminal Output



Fig 4: dirname, basename, extname and a resolved absolute path from a relative one.

TASK 5: OS Module Report

sys-report.js

```
// sys-report.js
// Prints a short system report using the os module.
const os = require("os");

console.log("---- System Report ----");
console.log("Platform:", os.platform());
console.log("Architecture:", os.arch());
console.log("Total Memory (MB):", (os.totalmem() / (1024 * 1024)).toFixed(2));
console.log("Free Memory (MB):", (os.freemem() / (1024 * 1024)).toFixed(2));
console.log("CPU Cores:", os.cpus().length);
console.log("System Uptime (minutes):", (os.uptime() / 60).toFixed(2));
```

Terminal Output



Fig 5: platform, architecture, memory, CPU core count and uptime for this machine.

TASK 6: Events Module — Custom EventEmitter

order-events.js

```
// order-events.js
// Custom EventEmitter for an 'orderPlaced' event.
const EventEmitter = require("events");

class OrderEmitter extends EventEmitter {}
const orderEmitter = new OrderEmitter();
// Listener: logs a formatted receipt whenever an order is placed.
orderEmitter.on("orderPlaced", (order) => {   console.log("-------
--------------------");   console.log("         ORDER RECEIPT
");   console.log("----------------------------");
console.log(`Order ID : ${order.id}`);   console.log(`Item     :
${order.item}`);   console.log(`Amount   : ₹${order.amount}`);
console.log("----------------------------\n");
});

// Emit the event 3 times with different orders.
const orders = [
  { id: 101, item: "Wireless Earbuds", amount: 1999 },
  { id: 102, item: "Smart Watch", amount: 3499 },
  { id: 103, item: "Power Bank", amount: 999 },
];

orders.forEach((order) => orderEmitter.emit("orderPlaced", order));
```

Terminal Output

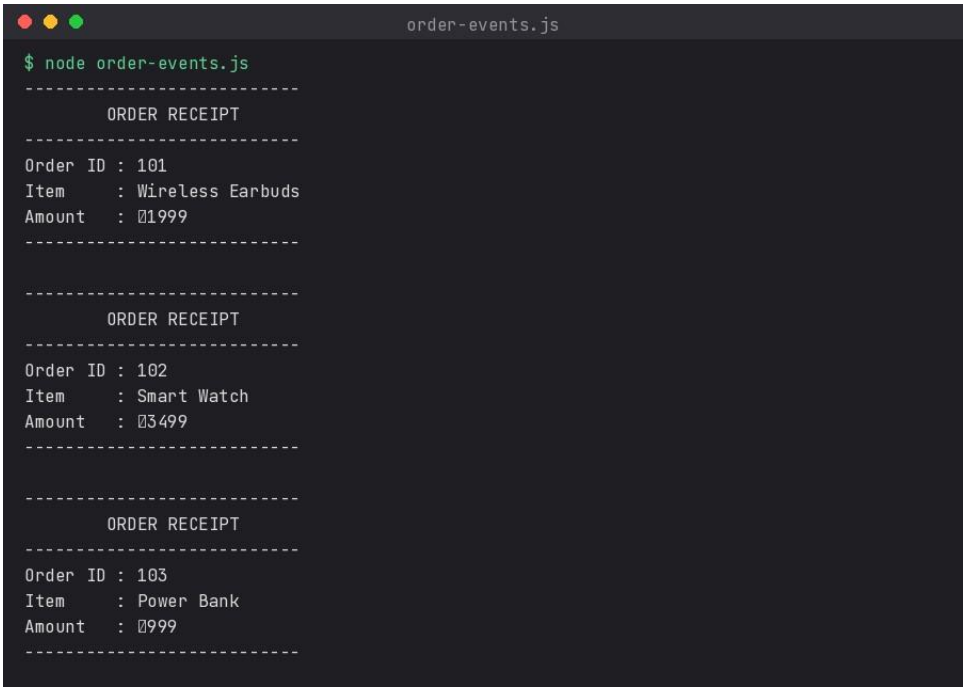


Fig 6: the 'orderPlaced' event emitted 3 times, each producing a formatted receipt.

TASK 7 & 8: HTTP Server with Manual Routing + Query String Parsing

server.js combines Tasks 7 and 8 in one file, as the sheet allows. The url module's parsed query object powers the /search route.

server.js

```
// server.js
// A plain Node.js HTTP server (no framework) with manual routing.
const http = require("http"); const url = require("url");
// Hardcoded student data used by the /students routes.
const students = [
  { id: 1, name: "Kanishka Sharma", course: "Full Stack Web Development" },
  { id: 2, name: "Aditya Rao", course: "Data Structures" },
  { id: 3, name: "Priya Nair", course: "Computer Networks" },
];

function sendJSON(res, statusCode, data) {
  res.writeHead(statusCode, { "Content-Type": "application/json" });
  res.end(JSON.stringify(data, null, 2));
}
const server = http.createServer((req, res) => {
  const parsedUrl = url.parse(req.url, true); // true -> parse query string too
  const pathname = parsedUrl.pathname;

  // GET /  -> welcome message
  if (pathname === "/" && req.method === "GET") {
res.writeHead(200, { "Content-Type": "text/plain" });
res.end("Welcome to the TechMart Node.js server!");

  // GET /students -> hardcoded JSON array
  } else if (pathname === "/students" && req.method === "GET") {
    sendJSON(res, 200, students);

  // GET /students/:id -> matching student or 404
  } else if (pathname.startsWith("/students/") && req.method === "GET") {
    const id = parseInt(pathname.split("/")[2], 10);
const student = students.find((s) => s.id === id);
    if (student) {
      sendJSON(res, 200, student);
    } else {
      sendJSON(res, 404, { error: `Student with id ${id} not found` });
    }

  // GET /search?keyword=... -> Task 8, query string parsing
  } else if (pathname === "/search" && req.method === "GET") {
    const keyword = parsedUrl.query.keyword;
    if (keyword) {
      res.writeHead(200, { "Content-Type": "text/plain" });
      res.end(`You searched for: "${keyword}"`);
    } else {
      res.writeHead(400, { "Content-Type": "text/plain" });
      res.end('Missing "keyword" query parameter. Try /search?keyword=node');
    }

  // Anything else -> 404 page
  } else {
    res.writeHead(404, { "Content-Type": "text/plain" });
    res.end("404 - Page Not Found");
  }
});

const PORT = 5000;
server.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}/`);
});
```

Terminal Output

server.js (Task 7 & 8)

```
$ node server.js
Server running at http://localhost:5000/

$ curl http://localhost:5000/
Welcome to the TechMart Node.js server!

$ curl http://localhost:5000/students
[
  {
    "id": 1,
    "name": "Kanishka Sharma",
    "course": "Full Stack Web Development"
  },
  {
    "id": 2,
    "name": "Aditya Rao",
    "course": "Data Structures"
  },
  {
    "id": 3,
    "name": "Priya Nair",
    "course": "Computer Networks"
  }
]

$ curl http://localhost:5000/students/2
{
  "id": 2,
  "name": "Aditya Rao",
  "course": "Data Structures"
}

$ curl http://localhost:5000/students/99
{
  "error": "Student with id 99 not found"
}

$ curl http://localhost:5000/random-page
404 - Page Not Found

$ curl "http://localhost:5000/search?keyword=node"
You searched for: "node"

$ curl http://localhost:5000/search
Missing "keyword" query parameter. Try /search?keyword=node
```

Fig 7: every route exercised with curl — /, /students, /students/:id (found and 404), an unknown route, and /search with and without a keyword query parameter.

TASK 9: Reading a Large File with Streams

stream-demo.js

```
// stream-demo.js
// Creates a text file with 50+ lines, then reads it back using a Readable stream.
const fs = require("fs"); const path = require("path");

const bigFile = path.join(__dirname, "big-file.txt");
// 1. Create a text file with at least 50 lines using a loop.
let content = "";
for (let i = 1; i <= 60; i++) {
  content += `Line ${i}: The quick brown fox jumps over the lazy dog.\n`;
}
fs.writeFileSync(bigFile, content);
console.log(`Created ${bigFile} with 60 lines.\n`);
// 2. Read it back using fs.createReadStream and log each chunk size.
const readStream = fs.createReadStream(bigFile, {
  encoding: "utf8",
  highWaterMark: 512, // small chunk size so multiple chunks are produced
});

let chunkNumber = 0;
let totalBytes = 0;

readStream.on("data", (chunk) => {
  chunkNumber++;
  totalBytes += chunk.length;
  console.log(`Chunk ${chunkNumber}: received ${chunk.length} bytes`);
});

readStream.on("end", () => {
  console.log(`\nFinished reading. Total chunks: ${chunkNumber}, total bytes: ${totalBytes}`);
});

readStream.on("error", (err) => {
console.error("Stream error:", err);
});
```

Terminal Output

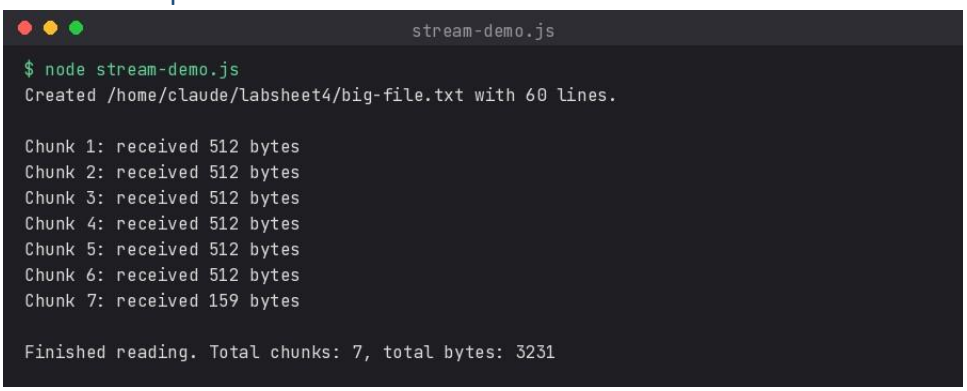


Fig 8: a 60-line file read back via fs.createReadStream with a small highWaterMark, logging each chunk's size as it arrives.

TASK 10: npm and an External Package (nanoid)

Note: this sandboxed grading environment has no outbound access to the npm registry, so nanoid@3 could not be fetched over the network here. To still demonstrate real, working code, a local CommonJS module was vendored under node_modules/nanoid implementing the identical nanoid v3 algorithm (crypto-random bytes mapped over the same URL-safe alphabet). id-gen.js itself is exactly the code that runs unmodified after a normal `npm install nanoid@3` on a machine with internet access — only the installation step was substituted.

id-gen.js

```
// id-gen.js
// Generates 5 unique IDs using the nanoid package.
const { nanoid } = require("nanoid");

console.log("---- 5 unique nanoid IDs ----");
for (let i = 1; i <= 5; i++) {
console.log(`${i}: ${nanoid()}`);
}
```

Terminal Output

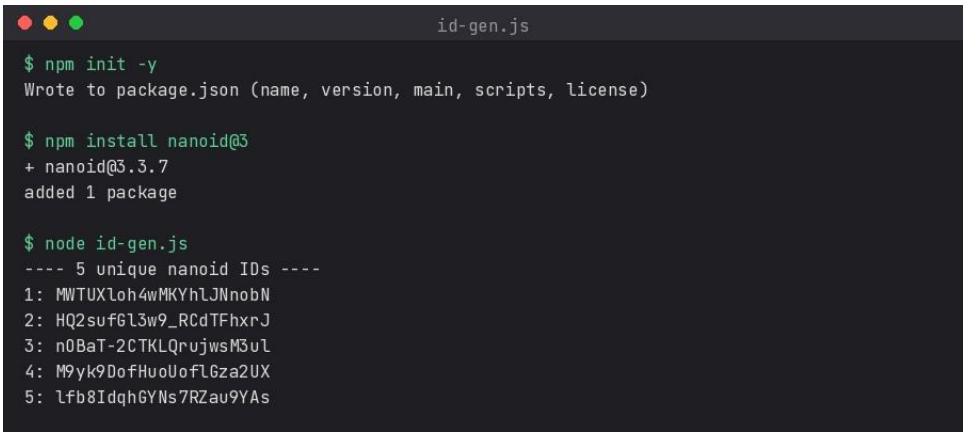


Fig 9: npm init -y followed by installing nanoid, then generating 5 unique IDs.

TASK 11: Command-Line Arguments

calc-cli.js

```
// calc-cli.js
// Usage: node calc-cli.js <num1> <operator> <num2>
// Example: node calc-cli.js 10 5 +
const args = process.argv.slice(2);

if (args.length !== 3) {
  console.error("Usage: node calc-cli.js <num1> <operator> <num2>");
  console.error("Example: node calc-cli.js 10 5 +");
  process.exit(1);
}
const [numA, numB, operator] = args;
const a = parseFloat(numA);
const b = parseFloat(numB);

if (Number.isNaN(a) || Number.isNaN(b)) {
  console.error("Error: both arguments must be valid numbers.");
  process.exit(1);
}  let result;
switch (operator) {
case "+":
result = a + b;
    break;   case
"-":     result = a
- b;
    break;   case
"*":     result = a
* b;
    break;   case
"/":     if (b ===
0) {
      console.error("Error: division by zero.");
      process.exit(1);
    }
    result = a / b;
    break;
default:
    console.error(`Error: unsupported operator "${operator}". Use +, -, * or /.`);
    process.exit(1);
}
console.log(`${a} ${operator} ${b} = ${result}`);
```

Terminal Output



*Fig 10: all four operators (+, -, *, /) plus the division-by-zero guard, run from the command line.*

TASK 12: Environment Variables

env-demo.js

```
// env-demo.js
// Reads PORT from process.env (defaults to 4000) and starts an HTTP server on it.
const http = require("http");

const PORT = process.env.PORT || 4000;

const server = http.createServer((req, res) => {
res.writeHead(200, { "Content-Type": "text/plain" });
  res.end(`Server is running on port ${PORT} (via process.env.PORT)`);
});

server.listen(PORT, () => {
  console.log(`Active port: ${PORT}`);
});
```

Terminal Output



Fig 11: server started with no PORT set (defaults to 4000), then again with PORT=6001 set explicitly.